The research paper I read: https://queue.acm.org/doi/10.1145/1365490.1365500

This was a very basic introduction to the CUDA programming model and how programs are executed.

It touched on the basic ideas of how programs are run and parallelised on a GPU. The three main components discussed are Individual threads, thread blocks, and grids. A grid is a set of thread blocks, where each block can be executed in any order, in parallel or in series. A thread block is a set of threads that also execute in parallel. Threads in a block have block private shared memory, which allows threads in the same block to communicate and access this shared memory.

When you execute a kernel, the programmer specifies the number of threads in a block, and the number of blocks making up the grid. Each thread has a unique ID within its thread block and each thread block has a unique ID within its grid. A thread block can have up to 512 threads.

the `__global__` specifier indicates this procedure is a kernel entry point.

An example call: kernel<<<dimGrid, dimBlock>>>(... param list …);

The values of `dimGrid` and `dimBlock` can be 1D - 2D - 3D vectors. The default is a 1D (int) value. The paper doesn't expand on why it supports up to 3D indexing, I assume this is to make it easier for 2D/3D indexing when working on those types of matrices.

If dimGrid=4 and dimBlock=256, inside of your kernel your index variables will have these ranges:

blockIdx.x = 0-3
threadIdx.x = 0-255
blockDim.x = 256
gridDim.x = 4

Meaning gridDim is 4 and blockDim is 256, blockIdx can be 0-3 because we have 4 blocks and threadIdx can be anywhere from 0 to 255 because each block has 255 threads. These values are used to index into your data structures to access the piece of memory a particular thread is responsible for.

Threads within a block run concurrently and have a shared memory region within that block. Threads can synchronize using _threadsync(). Synchronization is necessary for safe communication between shared memory regions. In this case synchronization is a barrier I believe.

Thread blocks execute independently. It must also be possible to execute thread blocks in any order, in parallel, or in series. This means blocks have no means of direct communication.

Although they can coordinate activity through global memory. This allows thread blocks to be scheduled in any order across any number of cores.

There are three main areas of memory a thread has access to. Thread local memory, block shared memory, and global memory. Each has different performance characteristics.

The rest of the paper goes over example application usage of CUDA and a few example implementations of things like a sparse matrix vector product and reductions.

I also read a section of this CUDA programming guide/intro to CUDA: https://docs.nvidia.com/cuda/cuda-programming-guide/part1.html

This goes much more in depth than the paper on the programming model. And introduces many more topics.

CPU and GPU code/memory is referred to as host memory and device memory. Code run on the GPU is referred to as a kernel and starting the kernel is referred to as launching the kernel.

A GPU can be thought of as a collection of Streaming Multiprocessors (SMs) which are organized into groups called Graphics Processing Clusters (GPC). Each SM has a local register file, Unified cache, and a number of functional units that perform computations.

All threads within a thread block are executed on a single SM. Grids can have millions of thread blocks, and each block can be scheduled on different SMs. Because threads from a block are executed on the same SM, this is how they are able to communicate since each SM has a shared memory region. This is also why threads between different blocks cannot communicate because there is no guarantee that all blocks in the same grid run on the same SM.

A cluster is a feature from CUDA 9.0 which allows you to group thread blocks. This ensures thread blocks will run on SMs within the same GPC.

Another new topic mentioned in this guide is warps.

Warps are within a thread block. Each thread block groups threads into a warp which is 32 threads. Warps execute using the Single Instruction Multiple Threads (SIMT) paradigm. This means all threads are executing the same instruction. The downside is that some threads in a warp may take different execution branches. When this happens it is referred to as warp divergence.

The solution here is to mask off the diverged threads and only execute the code for the threads executing the current kernel code. Then eventually the other threads will execute. It is better if all threads in a warp do not diverge for maximum utilization.

You should also always make your thread blocks a multiple of 32 to ensure maximum utilization of warps.

Finally this guide starts talking about memory models a little. The most interesting concepts being Unified memory and Unified virtual addressing (UVA). UVA means that all GPUs and CPUs on the system essentially share a virtual address space and are distinct and unique addresses from each other. This means you can tell which memory address belongs to which device or if its device or host memory.

Unified memory builds on top of this. Normally memory is either allocated by the device or the host. If it's allocated by the device only the device can access this memory and the same for host memory. With unified memory, you are able to allocate the memory in a way that the device and host can access the same memory. The data is transparently migrated between the two without the user having to do anything.